

# 用于 LLM 推理优化的 Workload–Router–Pool 架构

来自 vLLM Semantic Router 项目的愿景论文

Huamin Chen<sup>1</sup> Xunzhuo Liu<sup>1</sup> Bowei He<sup>2</sup> Fuyuan Lyu<sup>3,4</sup> Yankai Chen<sup>2</sup>

Xue Liu<sup>1,2,3,4</sup> Yuhan Liu<sup>5</sup> Junchen Jiang<sup>6</sup>

<sup>1</sup>vLLM Semantic Router Project, <sup>2</sup>MBZUAI, <sup>3</sup>McGill University, <sup>4</sup>Mila, <sup>5</sup>University of Chicago,  
<sup>6</sup>Tensormesh Inc. / UChicago

**Abstract.** 在过去一年中，vLLM Semantic Router 项目连续发布了一系列工作，覆盖以下方向：(1) 核心路由机制，包括信号驱动路由、上下文长度资源池路由、路由器性能工程、策略冲突检测、低延迟嵌入模型、类别感知语义缓存、用户反馈驱动的路由自适应、幻觉检测，以及用于隐私保护与越狱防护的分层内容安全分类；(2) 集群优化，包括集群配置与能效分析；(3) 智能体与多模态路由，包括多模态智能体路由、工具选择、CUA 安全，以及多轮上下文记忆与安全；(4) 治理与标准，包括推理路由协议和多提供方 API 扩展。这些论文都各自解决了 LLM 推理中的某个具体问题，但这些问题并不是彼此独立的；例如，集群配置依赖于路由策略，而路由策略又依赖于工作负载组成，并且这种组成会随着组织采用智能体与多模态工作负载而持续变化。

本文把这些结果提炼为 *Workload–Router–Pool (WRP)* 架构，这是一个面向 LLM 推理优化的三维框架。**Workload** 用来刻画资源池究竟在服务什么样的请求（聊天还是智能体、单轮还是多轮、暖请求还是冷请求、prefill 主导还是 decode 主导）。**Router** 决定每个请求应如何被分发（静态语义规则、在线 bandit 自适应、基于 RL 的模型选择、质量感知级联）。**Pool** 则定义推理在何处执行（同构还是异构 GPU、解耦的 prefill/decode、KV-cache 拓扑）。我们将既有工作映射到一个  $3 \times 3$  的 WRP 交互矩阵中，指出哪些单元已经被覆盖、哪些仍然开放，并提出二十一项位于这些交叉点上的具体研究方向；这些方向均以我们已有测量结果为依据，并按成熟度从工程可落地到开放研究问题进行分层。

Date: 2026 年 3 月



## 1 引言

决定生产环境 LLM 推理形态的有三个变量：工作负载、路由策略和 GPU 资源池架构。对它们的研究大多分别由彼此相对独立的社区展开，即工作负载刻画、模型路由以及系统/集群优化，这

些社区很少彼此交互。但在实际中，针对某一种工作负载和某一种资源池拓扑调优出来的路由决策，换到另一种组合上往往就不再最优。

我们的研究计划。vLLM Semantic Router (vLLM-SR) 项目 [1] 一直在构建推理路由栈。围绕不断扩展的工作体系（见 Table 1 与 Section 2），我们已经覆盖了四个支柱方向：

- **路由架构**：包括由信号驱动的语义路由及其可组合信号机制 [1]，面向概率式机器学习谓词的无冲突策略语言 [2]，通过 Flash Attention 与提示压缩实现的  $98\times$  路由器延迟下降 [3]，用于可配置延迟-质量权衡的二维 Matryoshka 嵌入 [4]，选择性推理路由 [5]，类别感知语义缓存 [6]，由用户反馈驱动的在线路由自适应 [7]，利用条件式事实核查分类与 token 级检测实现的幻觉感知响应校验 [8,9]，以及遵循 MLCommons AI Safety Taxonomy 的分层内容安全分类，用于隐私保护和越狱防护，其中包含二元安全/不安全门控 [10] 与九类危险分类器 [11]，从而支持按类别执行策略并实现保护隐私的路由；
- **资源池路由与集群优化**：包括令牌预算感知的资源池路由 [12]，带有网关层压缩的 FleetOpt 分析式集群配置 [13]，基于排队论的容量规划工具 inference-fleet-sim [14]，以及面向能效分析的  $1/W$  定律 [15]；
- **多模态与智能体路由**：包括多模态智能体路由 [16]、工具选择 [17]、CUA 安全 [18]，以及以网关为中心、支持多轮上下文记忆与安全控制的个人助理栈 OpenClaw [19]；
- **治理与标准**：包括 IETF 中提出的语义推理路由协议 SIRP [20]，以及面向智能体 AI 推理 API 的多提供方扩展 [21]。

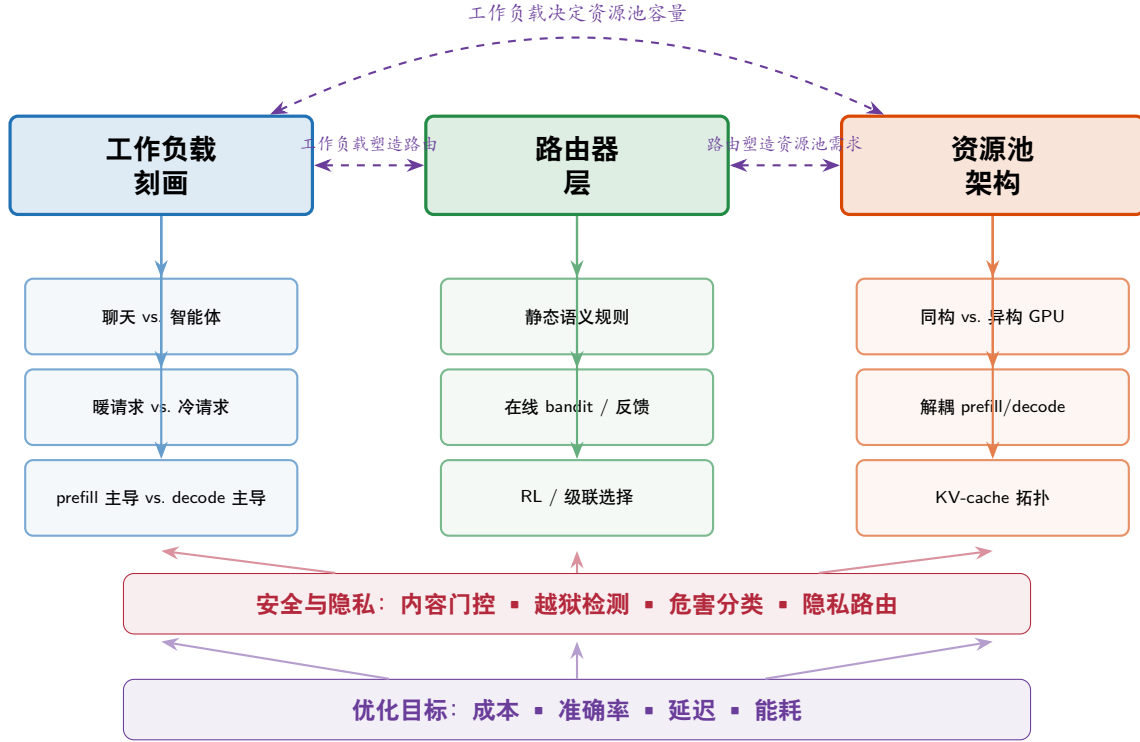
这些论文各自解决了一个具体问题，但这些问题彼此耦合。例如，资源池路由 [12] 会形成一个成本陡崖，而要通过网关层压缩来缓解它，则取决于工作负载提示长度 CDF 的原型结构 [13]。 $1/W$  定律 [15] 表明，相比 GPU 代际，路由拓扑才是更强的能耗杠杆，但前提是资源池支持按上下文长度进行划分。

我们的贡献。我们提出了 **Workload–Router–Pool (WRP) 架构** (Figure 1)，它是一个三维框架，用来组织我们已有的工作及更广泛的研究版图。本文的贡献如下：

1. 给出一个**结构化分解**，将问题分为 Workload、Router、Pool 三个相互作用的维度，这一分解建立在我们以往工作的证据之上。
2. 将既有工作**映射**到一个  $3 \times 3$  的 WRP 交互矩阵中，展示我们已经覆盖了哪些单元、哪些仍然空缺。
3. 提出一份包含二十一项具体机会的**研究路线图**，这些机会都位于不同维度的交叉处，并都以我们已有的测量结果为依据。

## 2 基础：vLLM Semantic Router 研究计划

Table 1 列出了支撑本文的那些论文。我们将它们分成四个支柱。



**Figure 1** Workload-Router-Pool (WRP) 架构。三条维度通过虚线双向箭头相互作用。安全与隐私是一个横切层：内容门控与危害分类会同时影响这三个维度。底部的优化目标则进一步约束整个系统。我们此前的工作覆盖了这一框架中的一个或多个单元。

## 2.1 支柱 1：路由架构

vLLM-SR 立场论文 [1] 提出了由信号驱动的决策路由：把异构信号，例如关键词模式、嵌入相似度、领域分类器和语言检测，组合成部署特定的路由策略，总共覆盖十三类信号。ProbPol [2] 形式化了当概率式机器学习信号同时触发时的冲突检测问题；mmBERT-Embed [4] 则提供了嵌入骨干，并通过二维 Matryoshka 结构支持可配置的延迟-质量权衡（见 Section 5.1）。

一个设计约束是，路由器必须同时满足低延迟、低资源占用：它不应要求专用 GPU，因为那个 GPU 本可以直接拿来承载 LLM 推理。FastRouter [3] 正是围绕这一点展开：面对长上下文请求，它把延迟降低了 98×，同时 GPU 占用低于 800 MB（见 Section 5.1）。

除请求发出时的分发以外，路由栈还包含响应时机制。WhenToReason [5] 负责判断某个查询是否需要进入 reasoning 模式。SemCache [6] 通过带有领域特定相似度阈值的类别感知语义缓存，消除重复推理（见 Section 4.2）。Feedback Detector [7] 使得系统可以根据每轮用户满意度信号进行在线路由自适应（见 Section 5.2）。HaluGate [8] 与 Factcheck Classifier [9] 进一步把路由扩展到响应阶段验证：它们把事实性查询导向更可靠的模型，并把每个模型的幻觉率反馈回路策略中（见 Section 5.1）。

该路由栈还通过遵循 MLCommons AI Safety Taxonomy 的分层分类流水线来执行内容安全与隐

**Table 1** 构成 WRP 愿景基础的 vLLM Semantic Router 项目论文。每一行都标明该论文主要涉及哪些 WRP 维度。**W**=Workload, **R**=Router, **P**=Pool。

| 简称               | 标题 / 发表渠道                                                    | W | R | P |
|------------------|--------------------------------------------------------------|---|---|---|
| PoolRouting [12] | Token-Budget-Aware Pool Routing (arXiv)                      | • | • | • |
| FleetOpt [13]    | Analytical Fleet Provisioning with C&R (arXiv)               | • | • | • |
| 1/W Law [15]     | Context-Length Routing and Energy Efficiency (arXiv)         | • | • | • |
| FeedbackDet [7]  | mmBERT-32K Feedback Detector (HuggingFace)                   | • | • |   |
| AVR [16]         | Adaptive VLM Routing for CUAs (arXiv)                        | • | • |   |
| MPExt [21]       | Multi-Provider Extensions for Agentic AI (IETF)              | • | • |   |
| OpenClaw [19]    | Personal assistant + Gateway (GitHub OSS)                    | • | • |   |
| SemCache [6]     | Category-Aware Semantic Caching (arXiv)                      |   | • | • |
| FleetSim [14]    | Queueing-Grounded Fleet Capacity Planner (arXiv)             | • |   | • |
| vLLM-SR [1]      | Signal-Driven Decision Routing for MoM Models (arXiv)        |   | • |   |
| ProbPol [2]      | Conflict-Free Policy for Probabilistic ML Predicates (arXiv) |   | • |   |
| FastRouter [3]   | 98× Faster LLM Routing (arXiv)                               |   | • |   |
| WhenToReason [5] | Semantic Router for vLLM (NeurIPS MLForSys)                  |   | • |   |
| mmBERT-Embed [4] | 2D Matryoshka Embeddings for Routing (HuggingFace)           |   | • |   |
| HaluGate [8]     | Token-Level Hallucination Detection (vLLM Blog)              |   | • |   |
| FactCheck [9]    | mmBERT-32K Factcheck Classifier (HuggingFace)                |   | • |   |
| OATS [17]        | Outcome-Aware Tool Selection (arXiv)                         |   | • |   |
| VCD [18]         | Visual Confused Deputy: CUA Security (arXiv)                 |   | • |   |
| SafetyL1 [10]    | MLCommons Binary Safety Classifier (HuggingFace)             |   | • |   |
| SafetyL2 [11]    | MLCommons 9-Class Hazard Classifier (HuggingFace)            |   | • |   |
| SIRP [20]        | Semantic Inference Routing Protocol (IETF)                   |   | • |   |

私控制。Level-1 二元分类器 [10] 对每个人站请求做一次安全/不安全判断；被标记为不安全的请求会升级到 Level-2 九类危害分类器 [11]，后者将其映射到具体类别（暴力犯罪、隐私、错误信息等）。这一两阶段设计让良性流量几乎不增加额外开销，同时仍能支持按类别执行策略，例如直接阻断侵犯隐私的提示、将易产生错误信息的查询导向事实性更强的模型、或者对专业建议类请求进行合规审计记录（见 Section 5.1）。

## 2.2 支柱 2：资源池路由与集群优化

令牌预算资源池路由 [12] 是起点：它根据估计的总令牌预算，将请求分发到短上下文池或长上下文池，在生产轨迹上减少了 17–39% 的 GPU 实例（见 Section 5.1）。

FleetOpt [13] 把资源池路由与网关层压缩统一到同一个分析框架中。给定工作负载 CDF 与延迟目标，它可以推导出最小成本的双池集群，以及最优边界  $B^*$  与最优压缩参数  $\gamma^*$ 。与之互补，inference-fleet-sim [14] 使用覆盖 A10G、A100 与 H100 的离散事件仿真器验证了这些结果（见 Section 6.1）。

1/W 定律 [15] 则量化了能耗维度：路由拓扑比 GPU 代际更能撬动能效，而且这种收益是可乘的（见 Section 6.2）。

## 2.3 支柱 3：多模态与智能体路由

Adaptive VLM Routing (AVR) [16] 把路由扩展到视觉语言 CUA 任务，提出了一个三层框架：第 1 层是多模态分类器（预路由约  $\sim 10$  ms），第 2 层是带 token 过滤评分的小型 VLM 置信度探测，第 3 层是大模型升级。AVR 预计可将成本降低多达 78%，同时与全程使用大模型的基线相比仅落后 2 个百分点以内。

OATS [17] 处理的是关键请求路径中的工具选择问题：通过离线方式把工具嵌入插值到“成功查询”质心方向，它在不增加服务时开销的情况下，将 MetaTool 上的 NDCG@5 从 0.869 提高到 0.940。

Visual Confused Deputy [18] 则把 CUA 感知错误重构为一种安全漏洞，并提出双通道对比分类方法，分别评估点击目标与推理文本，达到  $F1=0.915$ 。与 AVR 结合后，VCD 使得同一套路由流水线同时提供效率与安全。

## 2.4 支柱 4：治理与标准

在标准层面，Semantic Inference Routing Protocol (SIRP) [20] 在 IETF 中为 AI 推理系统中的内容级分类与语义路由规定了一套框架。Multi-Provider Extensions [21] 则把智能体 AI 推理 API 扩展到了多提供方环境。

# 3 WRP 架构

我们的论文体系支持如下论点：

**命题 1** (WRP 耦合性). *LLM* 推理优化中的三个维度，即 **Workload**、**Router** 与 **Pool**，彼此是耦合的，而不是正交的。若只孤立地优化某一个维度，就会留下大量效率红利没有被利用。最大的收益来自跨维度的联合优化。

来自我们自身工作的证据如下：FleetOpt [13] 表明，将路由压缩参数  $\gamma$  与资源池规模  $(n_s, n_l)$  共同设计，比在既有集群之上事后加压缩，成本还能再低 3.1–6.4%（即 **Router**  $\times$  **Pool** 耦合）。1/W 定律 [15] 表明，同一套 GPU 集群会因为所服务的上下文窗口不同而产生  $40\times$  的能效差异（即 **Workload**  $\times$  **Pool** 耦合）。AVR [16] 则说明，VLM 工作负载所需的路由信号与纯文本聊天完全不同（即 **Workload**  $\times$  **Router** 耦合）。

**定义 1** (WRP 分解). 一个生产级 LLM 推理部署可以由三元组  $(\mathbf{W}, \mathbf{R}, \mathbf{P})$  来刻画，其中：

- **W**：表示工作负载，即对请求类型、提示长度、生成长度、会话结构以及模态的分布；
- **R**：表示路由器，它将每个请求映射为一个 (model, pool, configuration) 三元组；以及
- **P**：表示资源池架构，规定 GPU 类型、资源池边界、解耦拓扑以及 KV-cache 管理方式。

优化目标是在 SLO 约束下，对成本（每请求 GPU 小时）、准确率（任务完成质量）、延迟（P99 TTFT、TPOT）和能耗（每瓦令牌数）进行加权组合。



## 4 维度 1：工作负载刻画

我们的集群优化工作 [12–14] 表明，工作负载刻画会驱动其余设计决策。我们沿着四条轴线来组织工作负载。

### 4.1 聊天 vs. 智能体工作负载

聊天工作负载（LMSYS-Chat-1M [22]，以及与 Splitwise [23] 一同发布的 Azure LLM Inference Trace）以短提示为主：80–95% 的请求低于 8K token，但向 64K+ 方向存在长尾。智能体工作负载（SWE-bench [24]，BFCL [25]）则呈现出另一种模式：随着工具输出不断累积，其提示长度会随轮次数确定性地增长。ServeGen [26] 报告说，相比聊天机器人，智能体每个用户请求会触发 3–10× 更多的 LLM 调用。

这两类工作负载都是多轮的，但它们的会话结构不同。聊天会话是对话式的：每一轮都隐含携带来自之前交互的上下文，用户可能在轮次之间切换话题、提供修正，或者逐步升级跨轮的对抗性试探。智能体会话则是任务驱动的：随着工具输出不断累积，上下文单调增长，并且会话在任务完成或失败时终止。像 OpenAI Responses API 这样的多轮 API，会显式编码这些模式，提供对话状态管理与逐轮优化，而这些都是单请求路由无法利用的。一个具备多轮感知的路由器之所以必须维护会话上下文，原因有二：KV-cache 亲和性，以及路由信号质量。先前的轮次暴露出用户意图、主题漂移、难度升级和安全违规，而这些都是单轮分类器会遗漏的。

我们的资源池路由论文 [12] 第一次明确展示了聊天/智能体区分如何创造路由机会：在同构 64K 集群中，那 80–95% 的短聊天请求实际上被过度供给了，导致 7/8 的 KV-cache 槽位被浪费。与之相对，智能体会话按轮次确定性增长这一特征意味着，基于会话感知的预测很可能优于基于类别的 EMA 路由。

记忆管理是隐藏的成本放大器。除去上下文本身的增长，多轮会话还要面对一个由记忆管理失败引出的复合成本问题。当 LLM 产生幻觉或者选错工具时，错误输出会进入对话历史，并在后续每一轮都被重新发送，组织会为这些降低了准确率的内容继续支付输入 token 成本。智能体要么容忍被污染的上下文，从而冒着进一步发生自我条件化错误的风险 [27]，要么必须额外发起 LLM 调用，对这些错误轮次进行摘要、压缩或移除，然后才能继续。AgentHallu [28] 对这一挑战做了基准测试：即便是前沿模型，在定位“哪一步导致了幻觉”上的准确率也只有 41.1%，而面对工具使用型幻觉时甚至会降到 11.6%，这使得有针对性的清理既不可靠，也往往迫使系统退回到昂贵的全上下文重摘要。

一份综合性的记忆综述 [29] 总结了智能体记忆的五类机制，即驻留在上下文内的压缩、检索增强存储、反思式自我改进、分层虚拟上下文，以及策略学习式管理；它们各自有不同的成本轮廓。另一项直接比较成本与性能的研究 [30] 表明，在 100K token 上下文长度下，即使启用 prompt caching，逐轮推理收费仍会随着上下文长度同比增长，而基于事实的记忆系统在大约十轮之后就会变得更便宜，这恰好处于智能体会话所处的区间。上下文压缩能够缓解这一问题：

ACON [31] 通过基于失败的指南优化, 在保持 95+% 任务准确率的同时, 将峰值 token 数减少 26–54%; Focus [32] 则在 SWE-bench 任务上通过自主智能体驱动的上下文剪枝, 节省 22.7% 的 token。问题在于, 这些方法都发生在智能体一侧, 只优化单个会话。这里存在一个根本限制: SAMULE [33] 表明, 单轨迹反思, 也就是单个智能体所能获得的唯一层级, 只会产生狭窄的错误修正; 而跨任务学习会在不同会话中聚合失败模式, 从而得到可迁移的洞见, 显著优于逐会话基线。CORRECT [34] 进一步强化了这一点: 多智能体错误会以相似结构跨请求重复出现, 而一个由既往会话提炼而来的在线错误模式缓存, 可将步骤级错误定位能力提高多达 19.8%。Mistake Notebook Learning [35] 也展示了同样的原则: 基于批量聚类的失败分析, 可以生成无需参数更新即可防止已知陷阱的通用性指导。

路由器在这里具备一种结构性优势, 可以直接利用这种跨会话可迁移性: 它观察来自所有租户、所有领域的数千个会话结果, 并把失败模式聚合到一个按 *(model, domain, failure-class)* 组织的表中 (机会 4)。与之相比, 单个智能体只能看到自己的会话, 样本量太少, 不足以建立可靠的失败统计, 也不足以发现领域特定的压缩策略, 例如“编码会话允许激进地剪掉前期失败工具轨迹; 医疗会话则必须保留完整上下文用于合规审计”。路由器把这类跨会话智能作为一个集群级服务立即提供给每个智能体, 连那些原本会遭遇冷启动的新智能体也能立刻受益。

网关原生助手与集群可观测性。OpenClaw [19] 提供了这一模式的一个具体开源实例, 即个人助手 + 本地优先网关: 用户通过多种消息渠道与智能体交互, 而一个 WebSocket Gateway 负责协调会话、工具执行 (浏览器、自动化、设备节点) 和模型调用。这类部署会产生冗长而杂乱的多轮轨迹, 包括工具结果、重试、渠道切换与上下文压缩事件, 而固定的学术基准往往捕捉不到这些现象。推理路由器位于这些网关发出的模型流量的热路径上, 因此它可以记录: 哪种模型、哪种工具表面、哪条安全路径与短会话或失控会话相关。跨租户聚合这些轨迹, 可以为离线 RL 与集群先验数据集提供输入, 而单个本地助手根本无法积累到这样的数据量 (机会 1, 机会 6)。反过来, 围绕 token 和轮次最小化而调优的路由策略, 也会反向影响运营者在网关上对默认模型和工具预算的选择。

代理层的指令、工具与记忆表面。ITR [36] 会在每个智能体步骤上检索最小化的 system instruction 片段和缩减后的工具子集; 在作者自己的受控智能体基准上, 它报告说每步上下文 token 数下降约 ~95%, 而且正确工具路由有明显提升, 因为一整套大型工具目录本身就可能吃掉约 ~90% 的上下文窗口。相同的机制也可以放到推理网关层实现, 即在请求到达模型前重写 tools 载荷和 system 块, 这样即便某些框架总是把完整工具目录一并发送, 也仍然能从集群级校准中受益。ToolScope [37] 在智能体一侧合并和过滤工具; SMART [38] 则表明, 抑制工具滥用可以同时改善成本和下游准确率。一个能够协调缩减、合并提示和预算信号 (BATS [39]) 的路由器, 可以避免在每个 SDK 里重复实现互不兼容的策略。最后, 记忆不只是“智能体记住了什么”, 更是下一次 *prefill* 要为哪些内容付费: MemCost 风格的分析 [30] 说明了何时事实存储优于原始上下文, 而会话感知调度器 (AgServe [40], Continuum [41]) 则把 KV-cache 生命周期与多轮结构当作一等资源。路由器天然适合在“完整历史、注入摘要、检索优先 *prefill*”之间做出选择, 以同时降低 token 数与错误累积, 而后者本来会拉长会话轮数 (机会 7)。

## 4.2 暖请求 vs. 冷请求

暖请求会命中来自前一轮或共享系统提示的 KV-cache 前缀；冷请求则从零开始。缓存 token 的成本最高可降低  $10\times$  [42]，而在分布式部署中，带有前缀缓存感知的调度甚至可实现  $57\times$  更快的响应时间 [42]。

对于智能体工作负载而言，会话时间的 40–60% 都花在被暂停的工具调用上，这会让 LRU 驱逐把缓存命中率摧毁掉 [43]。会话亲和路由，也就是尽量把一个会话保留在同一资源池实例上，可以保住 KV-cache 局部性。更高层面上，类别感知语义缓存 [6] 可以与之互补：它依靠领域感知的相似度阈值和 TTL，直接消除冗余推理请求，也就是对语义等价的查询直接返回缓存响应，而不再调用 LLM，从而同时降低成本和资源池负载。

## 4.3 Prefill 主导 vs. Decode 主导工作负载

视觉语言模型会把图像编码成数百个视觉 token，因此 VLM 工作负载属于 *prefill* 主导。我们的 AVR 系统 [16] 正面处理了这个问题：CUA 截图约为  $\sim 5$  MB，而对 4K 截图做路由，要求路由层本身先抽取多模态信号。FastRouter [3] 则表明，传统 NLP 压缩可以把路由器输入限制在大约  $\sim 512$  token，无论原始提示多长，这使得 *prefill* 主导工作负载对路由层而言也变得可处理。

编码智能体则属于 *decode* 主导：代码生成会产生长而结构化的输出。*decode* 比例会直接影响解耦 *prefill/decode* 架构 (Splitwise [23]、DistServe [44]) 是否有收益，而我们的 *inference-fleet-sim* [14] 明确建模了这一权衡。

## 4.4 工作负载身份信号

我们先前的工作把入站请求当作不透明文本，并通过内容分析来推断工作负载类型，例如嵌入相似度、关键词匹配和领域分类器。实际上，请求结构本身就携带了丰富的身份信号，路由器无需检查内容就能利用它们，从而打开零延迟工作负载分类的可能。

显式的 API 层信号。现代推理 API 已经暴露出能够区分智能体与聊天工作负载的结构化元数据：

- **工具定义。**带有 `tools` 数组和函数 `schema` 的请求几乎一定是智能体工作负载；工具的数量和类型（代码执行、Web 搜索、MCP 连接器）还能进一步标识智能体所处的操作模式。工具目录本身就是一个成本杠杆：ITR [36] 表明，动态地只暴露每一步所需的最小工具子集，能够让每步上下文 token 减少 95%，同时使正确工具路由提高 32%，因为大型工具目录最多可消耗 90% 的可用上下文。
- **会话链。**像 `previous_response_id` (OpenAI Responses API) 或对话状态对象这类字段，表示这是多轮续接。它们的存在告诉路由器该请求位于会话中段（因此可以启用 KV-cache 保留，见 机会 15），同时无需解析整个对话历史就能知道当前轮次编号。
- **系统提示指纹。**系统提示通常在同一部署中是静态的，因此可在网关层做哈希。一个已知



哈希查找表能够把每个部署映射到其领域、预期工具集和历史会话长度分布，从而让路由器以零分类成本拿到每请求先验。

- **网关元数据头。**生产网关（如 Cloudflare AI Gateway 的 `cf-aig-metadata`、LiteLLM 的 `x-litellm-tags`，以及自定义 `X-Client-Request-Id` 头）已经允许运营者用环境、团队和工作负载类型标签来标注请求。SIRP [20] 则把这种做法标准化为分类轴头字段。

调用者身份与授权范围。上面四类信号描述的是请求是什么。但它们都没有描述是谁发起了这个请求，也没有描述他/它被授权做什么。每个 LLM API 调用里携带的 bearer token 恰好填补了这一空白：在生产网关中，它同时编码了调用者身份以及授权范围，例如某个 key 能访问哪些模型、能调用哪些工具、能消耗多少预算。

- **按 key 的工具权限。**LiteLLM 的 Tool Permission Guardrail [45] 使用正则匹配工具名和参数，为每个 key 执行工具的 allow/deny 规则。它提供两种模式，即 *block*（拒绝请求）与 *rewrite*（在模型看到之前剥离掉不允许的工具），这说明在网关层做“按调用者范围治理工具”的能力已经具备生产可用性。
- **结构化的智能体授权。**Agent Authorization Profile(AAP) [46] 是一个正在出现的 OAuth 2.0 扩展，它增加了五个 JWT claim：智能体身份、带有服务端约束的能力（按工具的域名限制、速率限制、时间窗口）、任务绑定（防止范围漂移）、委派跟踪以及审计轨迹。它的核心原则是：“授权必须在执行时评估，而不能只在连接建立时评估。”
- **基础设施类比。**Kubernetes RBAC 结合 admission webhook，会在 API server 边界依据身份强制执行权限，而不管 Pod 里的应用自己认为自己应该做什么。这正是同样的零信任原则 [47] 在 LLM 请求上的体现。

bearer token 使一种当前 WRP 路线图里尚未出现的新型路由决策成为可能：基于角色的工具调用强制执行（[机会 8](#)）、按调用者的全局信誉评分（[机会 9](#)），以及由路由器强制执行的行为承诺（[机会 10](#)）。它也扩展了已有机会：按调用者跨会话累积风险（[机会 3](#)）、跨会话 token 预算（[机会 5](#)）、治理 DSL 中的 RBAC 谓词（[机会 20](#)）、按角色调整工具目录（[机会 6](#)），以及感知调用者的记忆隔离（[机会 7](#)）。

推断得到的结构信号。除显式元数据外，路由器还可以在分发时从请求结构中推断工作负载身份：

- **对话深度。**`messages` 数组里消息对象的数量揭示了会话成熟度：一个只有 2 条消息的交换大概率是冷启动聊天；一个包含 20 条消息、且 `assistant/tool` 角色交替出现的数组，则更可能是一个深层智能体会话，并对应不同的模型、资源池和压缩需求。
- **工具调用历史模式。**如果此前的 `assistant` 消息里包含 `tool_calls`，并且后续 `tool` 角色消息报告了较高失败率，那么路由器可以在下一轮之前主动切换到更强模型，或者先做压缩。
- **推理元数据。**NoiT [48] 表明，chain-of-thought 推理步数的多少能在推理前预测任务难度，并据此实现难度感知模型选择，同时达到 95% 的对抗检测准确率。

从静态路由走向分阶段路由。这些信号使系统可以从“每个 API 调用做一次决策”的逐请求路由，转向“在智能体工作流内部按阶段做决策”的逐阶段路由。Aragog [49] 已经展示了这一点：它把一次性的、保持准确率的配置步骤与依赖系统当前观测的逐阶段调度器分离开来，因此在峰值负载下实现了 50–217% 的吞吐提升和 32–79% 的延迟下降。WRP 框架非常适合支持这一演化：路由器已经为了多轮感知而维护会话状态（VCD [18]，Feedback Detector [7]），而上面这些按阶段划分的身份信号，则为一个阶段感知的路由策略提供了输入特征，使其能随着会话演进而不断调整模型选择、压缩策略和资源池分配。

## 4.5 工作负载 CDF 原型

FleetOpt [13] 识别出三类提示长度 CDF 原型，它们决定了最优资源池与路由配置：

1. **Concentrated-below**（例如 Azure）：80–95% 都是短请求。此时激进压缩最能降低长池成本。
2. **Dispersed**（例如 LMSYS）：请求跨越整个长度谱。适合采用中等压缩的平衡型双池拆分。
3. **Concentrated-above**（例如 SWE-bench）：多数请求都很长。此时应提升  $\mathcal{B}_{\text{short}}$ ，而不是压缩。

这些原型并不是静态的：随着组织从聊天转向智能体，CDF 会从 Archetype 1 向 Archetype 3 迁移。FleetOpt [13] 表明，每种原型对应的最小成本集群在性质上都不同；inference-fleet-sim [14] 进一步确认，当  $\mathcal{B}_{\text{short}} < 4\text{K}$  时，对 Archetype 1 而言，单体拓扑甚至可能优于解耦拓扑。

## 5 维度 2：路由器层

路由器位于客户端与推理资源池之间。我们的论文覆盖了三类自适应机制：静态规则、在线 bandit 自适应，以及基于强化学习的选择。

### 5.1 静态语义路由

vLLM-SR 立场论文 [1] 建立了由信号驱动的架构：把十三类信号组合成适配具体部署的策略，并覆盖 vLLM、OpenAI、Anthropic、Azure、Bedrock 与 Gemini 等后端。嵌入相似度信号由 mmBERT-Embed [4] 提供，它的二维 Matryoshka 设计允许运营者按部署需要调节延迟与质量（第 6 层时为 2.56 ms，适合实时路由；第 22 层时为 11 ms，适合最大召回）。ProbPol [2] 则形式化了这些信号的冲突检测问题。

令牌预算资源池路由 [12] 实现了一种专门的静态规则：根据估算的总令牌预算，把请求分发到短池或长池，从而减少 17–39% 的 GPU。OATS [17] 针对工具选择问题，通过零额外服务成本的嵌入优化，将 NDCG@5 从 0.869 提升到 0.940。

WhenToReason [5] 另外负责判断某个查询是否值得进入 reasoning 模式，从而避免简单问题上不必要的 chain-of-thought token。Factcheck Classifier [9] 又添加了一种预路由信号：被判断为

事实查询的请求，可以被导向幻觉率更低的模型，或者在响应后交给 HaluGate [8] 做进一步验证。

对于多轮会话，vLLM-SR 项目 [1] 除了单请求分类以外，还提供了两种机制。第一种是上下文记忆注入：路由器会累积前几轮中的上下文信号，例如主题、领域、用户画像与难度变化轨迹，并在后续轮次的路由决策中注入这些上下文，从而让推理端点拿到更丰富的背景并提升回答准确率。这一点在精神上类似于 OpenAI Responses API 这类多轮 API 中的会话状态管理。第二种是面向多轮安全的对比嵌入分类：路由器不是孤立地判别每一轮，而是利用 few-shot 对比嵌入来检测跨多轮分散投放对抗内容的越狱尝试，而这类攻击是单轮分类器无法发现的。

内容安全与隐私又构成了另一道预路由门控。MLCommons Safety Level-1 分类器 [10] 对每个人站请求进行二元安全/不安全检查；被标记的请求会传递给 Level-2 危害分类器 [11]，后者从九个 MLCommons 类别中选出其一（暴力犯罪、非暴力犯罪、性犯罪、武器/CBRNE、自残、仇恨、专业建议、隐私、错误信息）。这种分层设计让安全流量的额外延迟几乎可以忽略不计（只有不安全请求才支付第二阶段开销），同时也使按类别进行路由成为可能，例如把侵犯隐私的提示在网关直接阻断，把错误信息查询导向事实依据更强的模型，并把专业建议请求记录下来用于合规审计。这两个分类器都使用 mmBERT LoRA 适配器，符合本项目对低资源占用的设计约束。

FastRouter [3] 保证所有这些路由决策，无论是单轮还是多轮，都能在延迟预算内完成：端到端 50 ms，GPU 占用 800 MB，小到足以与服务一起共置。

## 5.2 面向聊天的在线自适应路由

聊天工作负载天然带有一种反馈信号：用户的下一条消息会表明上一条回复是否令人满意。我们的 mmBERT-32K Feedback Detector [7] 把每个用户轮次分成四类（满意、需要澄清、答案错误、想换一种方式），准确率达到 98.8%，额外开销低于 1 ms。当检测器判断用户不满意时，路由器可以把后续轮次改派给更强的模型或不同的资源池；这里不需要显式奖励模型，因为用户自身的反应就构成了奖励。再结合路由器的上下文记忆注入机制（Section 5.1），这种自适应就变成了会话感知的：路由器不只是知道用户不满意，还知道是哪个主题和哪个难度层级触发了不满意，因此可以执行有针对性的重路由，而不是一刀切地升级模型。

在我们的项目之外，RouteLLM [50] 使用人类偏好数据训练路由器（以 26% 的成本达到 95% 的 GPT-4 质量）；MixLLM [51] 使用带查询标签的上下文 bandit（以 24.18% 的成本达到 97.25% 的 GPT-4 质量）；SageServe [52] 则把流量预测与基于 ILP 的路由结合起来（在微软规模下降低 25% 的 GPU 小时）。

## 5.3 面向智能体的 RL 路由

智能体工作负载不像聊天那样拥有逐轮用户反馈：“用户”往往是另一个 LLM 或编排器，而任务成功只有在多步会话结束时才能观测到。这样的延迟且稀疏的奖励，使 RL 比 bandit 更合适。Router-R1 [53] 展示了这一思路：它把路由器本身实例化为一个 LLM，在内部推理的“think”动作与模型调用的“route”动作之间交替，并用 PPO 端到端训练，奖励函数同时包含格式、结果

与成本,从而使路由器无需手工启发式就能学到性能-成本权衡。R2-Router [54] 联合选择模型和输出长度预算,把成本降低到原来的 1/4 到 1/5。M-CMAB [55] 则在异构预算下,对多模态调度使用在线 RL。

我们的 AVR 系统 [16] 把级联机制应用到了 VLM 智能体任务:第 1 层是多模态分类器(约 ~10ms),第 2 层是带 token 过滤评分的小型 VLM 置信探测,第 3 层是大模型升级。它预计可把成本降低到多达 78%,同时与全大模型基线相比仍保持在 2 个百分点以内。当它与 Visual Confused Deputy 防护 [18] 结合时,高风险动作会直接升级到最强模型。

自适应机制取决于工作负载。对聊天来说,由 Feedback Detector [7] 这类分类器检测到的逐轮用户反馈,提供了一种快速、低开销的在线调整信号。对智能体来说,只有在多步会话结束后才能看到任务级结果,因此更适合使用基于 RL 的优化 [53],以对一系列路由决策进行推理。

## 5.4 智能体循环、记忆与集群级编排

近期的智能体服务研究指出,面向工具使用型智能体的推理服务应被视为一个数据系统问题,而不是一串互不相关的聊天补全。Helium [56] 围绕 workflow 结构、复用和算子放置重构了智能体 LLM 服务;Sutradhara [57] 对工具密集型推理中的编排器和执行引擎做联合设计;CONCUR [58] 把拥塞感知批处理用于高吞吐智能体推理;AgServe [40] 表明会话感知调度能突破静态成本-质量权衡;Continuum [41] 则把多轮调度和 KV-cache 的生存时间耦合起来,避免被暂停的工具调用悄悄破坏局部性。这些系统与基于经验的路由(EvoRoute [59])以及预算感知顺序路由(BAAR [60])是对齐的,但它们通常都只在单一栈内部进行评估。

语义路由器对它们形成补充,因为它导出了一个全局集群级控制面:同一套信号驱动的模式路由钩子,可以同时做到:(i) 按 ITR 风格 [36] 重塑工具与指令载荷;(ii) 附加记忆层决策,例如全量上下文、摘要、或以检索优先的 prefill,这一过程受 MemCost [30] 与长程智能体上的 provider prompt-caching 研究 [61] 启发;(iii) 发出调度提示,如延后接纳、批处理亲和,从而减少由排队诱发的重试,而后者常常是额外智能体轮次的隐藏来源。xRouter 的 RL 编排 [62] 说明了,只要奖励定义在完整轨迹之上,路由层就能学到成本感知策略;如果再把这种能力与 OpenClaw [19] 这类网关原生栈结合,其中真实用户通过单一控制平面驱动高方差的多轮流量,那么可用于学习的轨迹多样性就会成倍增加,而无需所有框架共用一个记忆系统或工具 SDK。

## 6 维度 3: 资源池架构

我们的集群优化工具 [13-15] 为资源池架构提供了分析模型与仿真模型。

### 6.1 同构 vs. 异构 GPU 资源池

FleetOpt [13] 从分析上推导最小成本集群:它给出了满足边际 GPU 成本相等条件的双池架构及其最优边界  $B^*$ 。在生产轨迹上,相比同构集群,这一方法可将 GPU 成本降低 6-82%。inference-



fleet-sim [14] 又把这一点推广到异构 GPU 类型 (A10G、A100、H100)，并引入了物理启发的性能模型。

Mélange [63] 把异构 GPU 分配建模为成本感知的 bin packing，最高可实现 77% 的成本下降。我们的仿真工具 [14] 则表明，最便宜的 GPU 类型会以非直观的方式依赖于工作负载：对于 concentrated-below 类工作负载，并发性比单 token 速度更重要，因此 A10G 甚至可能优于 H100。

## 6.2 能耗与 1/W 定律

1/W 定律 [15] 对资源池架构有直接含义：上下文窗口每翻倍，每瓦令牌数就会减半，因为 KV-cache 并发能力受限。在 64K 上下文时，一块 H100 只能容纳 16 条序列 ( $\text{tok}/W \approx 1.5$ )；而在 4K 时则可以容纳 256 条序列 ( $\text{tok}/W \approx 17.6$ )。

路由拓扑是比 GPU 代际更强的能耗杠杆：双池路由相较同构集群能带来大约  $\sim 2.5\times$  更高的  $\text{tok}/W$ ，而 H100→B200 的升级只有大约  $\sim 1.7\times$ 。两者收益可以相乘，达到大约  $\sim 4.25\times$ 。对像 Qwen3-235B-A22B 这样的 MoE 模型而言，活动参数权重流式传输还提供了第三个杠杆（在 8K 上下文时约为  $\sim 37.8 \text{ tok}/W$ ，比稠密版 Llama-3.1-70B 高  $5.1\times$ ）。

因此，能耗优化要求联合设计路由与资源池；无论 GPU 代际如何，同构集群都无法到达能效前沿。

## 6.3 解耦的 Prefill/Decode 与 KV-Cache 拓扑

Splitwise [23] 与 DistServe [44] 在物理上把 prefill 与 decode 分离，在同等 SLO 下可服务  $4.48\times$  更多请求。Mooncake [64] 提供了以 KV-cache 为中心的解耦，并支持分层卸载（GPU HBM、CPU DRAM、SSD）。NVIDIA Dynamo [65] 则支持跨多级内存的 KV-cache 卸载与动态调度。

我们的 inference-fleet-sim [14] 对这三种拓扑都建模了，即单体式、双池路由式和解耦式，并揭示出一个结论：解耦并不总是有利。对于 concentrated-below 工作负载 (Archetype 1) 且  $B_{\text{short}}$  较小时，KV-cache 传输开销会超过它带来的收益。

vLLM 的 PagedAttention [66] 把 KV-cache 浪费降低到 4% 以下，而带有前缀缓存感知的分布式调度 [42] 则带来了  $57\times$  的加速。但对智能体工作负载而言，标准 LRU 驱逐是病态的 [43]，需要使用带会话亲和路由的上下文感知保留策略。

## 7 跨维交互：来自我们工作的经验

如果每个维度都被单独研究，跨维交互就很容易被忽略，但现实世界中的性能往往正是由它们主导的。下面我们形式化地总结我们的论文已经识别出的主要交互关系。



## 7.1 Workload × Router

**观察 1** (工作负载决定路由器的自适应机制). 聊天工作负载会产生逐轮用户反馈; 我们的 *Feedback Detector* [7] 能实时判断满意度, 从而使路由器在不建立显式奖励模型的情况下, 基于不满意信号进行重路由。智能体工作负载则没有这种逐轮信号, 任务成功只有在多步会话末尾才可观测, 因此基于 *RL* 的路由 [53] 更合适, 因为它能够对顺序决策进行推理。自适应机制必须与工作负载的反馈粒度相匹配。

**观察 2** (模态决定路由信号). 纯文本聊天使用嵌入相似度和关键词信号 [1]。VLM 工作负载则需要多模态分类器, 例如 AVR [16] 中的 *SigLIP+MiniLM*, 因为纯文本信号无法评估截图复杂度。路由器的信号集合必须与工作负载的模态相匹配。

**观察 3** (工作负载决定路由中的安全与隐私需求). 不同类型的工作负载, 会对路由器施加不同的安全与隐私要求。聊天路由错误会带来质量下降; 而多轮聊天还存在一个更隐蔽的威胁, 即对抗性用户可以把越狱内容分散到多轮中, 从而绕过单轮分类器。vLLM-SR [1] 通过对比嵌入分类来处理这一点, 它评估的是轮次序列而不是单独的消息。CUA 路由错误则会造成真正的安全漏洞: *Visual Confused Deputy* [18] 表明, 把某个 CUA 动作错误地路由给能力较弱的模型, 会导致 *grounding* 错误, 并可能被利用以完成权限提升。对于所有工作负载类型, 我们的分层 *MLCommons* 安全流水线, 即先二元门控 [10]、再九类危害分类 [11], 在路由层提供了统一的内容过滤, 而且精度细到类别级, 这使运营者可以执行工作负载特定的策略, 例如阻断面向客户聊天中的隐私泄露提示, 或在知识检索智能体中标记错误信息。安全与隐私应该被纳入路由目标之中, 其中聊天需要多轮感知, 智能体则需要动作级防护。

**观察 4** (事实型工作负载需要响应时验证). 仅靠请求时路由并不能保证正确性: 某个模型也许适合该查询所属的领域, 却仍可能在某个具体事实上产生幻觉。Factcheck Classifier [9] 在请求阶段识别事实查询提示 (约  $\sim 12ms$ ), HaluGate [8] 则在响应阶段以 *token* 级粒度进行验证 (额外开销为  $76-162ms$ )。两者共同闭合了一个仅靠请求时路由无法闭合的回路: HaluGate 观察到的每模型幻觉率会反馈回路路由策略, 逐步把事实流量导向更可靠的模型。

## 7.2 Router × Pool

**观察 5** (路由策略共同决定资源池规模). FleetOpt [13] 表明, 将压缩参数  $\gamma$  与资源池规模 ( $n_s, n_l$ ) 联合设计, 比在既有资源池上事后加压缩, 成本还可再降 3.1-6.4%。路由器不是覆盖在资源池之上的一个附属层, 而是联合设计变量本身。

**观察 6** (路由延迟约束资源池拓扑). FastRouter [3] 表明, 路由延迟是一个关键约束: 在  $4,918ms$  的基线下, 路由开销本身就超过了短上下文请求的 *TTFT SLO*。只有在把延迟降到  $50ms$ 、即完成  $98\times$  加速之后, 资源池路由才真正适用于整个请求流。mmBERT-Embed [4] 进一步强化了这一点: 它的二维 *Matryoshka* 设计使运营者可以用质量换延迟 (第 6 层  $2.56ms$ , 而第 22 层  $11ms$ ), 从而即便资源池拓扑变得更复杂, 相似度信号仍能留在路由延迟预算内。

**观察 7** (会话中途切换模型会产生 KV-cache 沉没成本). 在多轮会话中, 如果 *Feedback Detec-*

**Table 2** WRP 交互矩阵。每个单元格列出我们已经覆盖该跨维交互的论文，并附上关键外部参考。稀疏单元表示开放机会。

|                 | Workload                                    | Router                                                                                                                                      | Pool                                                                         |
|-----------------|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Workload</b> | CDF 原型 [13]; 轨迹 [23, 26]                    | AVR [16] (模态); VCD [18] (安全); SafetyL1/L2 [10, 11] (内容安全); FeedbackDet [7] (聊天自适应); HaluGate [8] (事实验证); 仍较稀疏, 见 <a href="#">机会 1 and 4</a> | FleetOpt [13] (CDF → 资源规模); 1/W Law [15] (能耗); 仍较稀疏, 见 <a href="#">机会 16</a> |
| <b>Router</b>   | FleetOpt [13] (压缩改变 CDF); WhenToReason [5]  | vLLM-SR [1] (信号); ProbPol [2] (冲突); FastRouter [3] (延迟); mmBERT-Embed [4] (嵌入); OATS [17] (工具); SemCache [6] (缓存); HaluGate [8] (验证)        | FleetOpt [13] (联合设计 $\gamma$ ); 仍较稀疏, 见 <a href="#">机会 11 and 15</a>         |
| <b>Pool</b>     | FleetSim [14] (拓扑 → 成本); 1/W [15] (拓扑 → 能耗) | 仍较稀疏, 见 <a href="#">机会 12</a>                                                                                                               | FleetSim [14] (A10G/A100/H100); M lange [63]; DistServe [44]                 |

tor [7] 或上下文记忆信号表明存在更好的模型，路由器就会面临一个两难局面。若在会话中途切换模型，就必须丢弃当前模型已经积累的 *KV-cache*，这是一种与会话长度和上下文规模成正比的沉没成本。若不切换，则保住了缓存投资，但放弃了质量提升，而这种机会成本会随着后续继续使用次优模型的轮次增加而上升。最优策略取决于剩余会话长度、模型之间的质量差距以及重建 *KV-cache* 的成本，因此会形成一条沉没成本与机会成本之间的 *Pareto* 前沿。当前还没有系统显式优化这一权衡。

### 7.3 Workload × Pool

**观察 8** (工作负载原型决定最优拓扑). 我们的集群工具 [13, 14] 表明: *Archetype 1* (*concentrated-below*) 最适合双池路由; *Archetype 3* (*concentrated-above*) 则可能受益于解耦的 *prefill/decode*, 因为大多数请求都很长; *Archetype 2* (*dispersed*) 可能需要多层资源池。

**观察 9** (智能体普及会推动能效前沿变化). *1/W* 定律 [15] 预测, 当工作负载从聊天转向智能体时, 除非资源池拓扑也同步调整, 否则整个集群的能效会下降 2–4×。双池路由可以收回大部分损失, 但前提是资源池边界能跟随工作负载演化。

## 8 将我们的论文映射到 WRP 矩阵

Table 2 将我们的每篇论文映射到 WRP 交互矩阵中, 展示它们各自覆盖了哪些跨维度单元。那些仍然稀疏的非对角单元, 正对应着 Section 9 中提出的开放机会。

**Table 3** 二十一项拟议机会的成熟度与可行性。**Tier** 表示主要障碍属于: *engineering* (构件已经存在, 主要欠缺集成) 还是 *research* (仍有开放技术问题)。

| #  | 机会                                 | Tier            | 主要障碍                                      |
|----|------------------------------------|-----------------|-------------------------------------------|
| 2  | HaluGate 模型信誉路由                    | Engineering     | 每个 (模型, 领域) 单元的样本量                        |
| 5  | 面向智能体的令牌预算约束                       | Engineering     | 领域特定预算模型的校准                               |
| 8  | Follow-the-sun 资源池再平衡              | Engineering     | 滑动窗口 CDF 估计 + FleetOpt                    |
| 12 | KV-cache 保留指令                      | Engineering     | 保留指令 API 集成                               |
| 15 | 治理即代码                              | Engineering     | 用 WRP 约束扩展 DSL                            |
| 19 | 请求级 RBAC 强制执行                      | Engineering     | 网关钩子 + JWT 集成                             |
| 1  | 面向会话长度路由的离线 RL                     | Research        | credit assignment + off-policy evaluation |
| 3  | 累积式多轮风险评分                          | Research        | 对抗基准上的阈值校准                                |
| 4  | 失败类别感知路由                           | Research        | 多维失败的加权                                   |
| 17 | <b>集群学习的联合工具-模型优化</b>              | <b>Research</b> | <b>联合 (tool, model) 结果表的稀疏性</b>           |
| 18 | 网关协调的智能体循环                         | Research        | 端到端集群集成尚未被整体评估; 子系统分别已验证                  |
| 20 | 全局调用者信誉                            | Research        | 异构安全事件评分 + 衰减校准                           |
| 21 | 路由器强制的行为承诺                         | Research        | 边界声明 NLU 分类器 + 误报风险                       |
| 6  | 基于可观测量的资源池状态推断                     | Research        | prefill 速率回归 + cold start                 |
| 7  | 输出长度感知资源池路由                        | Research        | 输出长度预测方差                                  |
| 9  | 资源池感知级联                            | Research        | 级联中止阈值 + grace period                     |
| 10 | 在线 $(\gamma, \mathcal{B}^*)$ 联合自适应 | Research        | 振荡控制 + 验证缺口                               |
| 11 | 混合原型集群配置                           | Research        | 面向多租户的 FleetOpt 扩展                        |
| 13 | 原型驱动的主动扩缩容                         | Research        | 原型感知预测 vs. 聚合基线                           |
| 14 | 把能耗作为路由目标                          | Research        | 按资源池配置校准 tok/W 信号                         |
| 16 | 闭环自适应                              | Research        | 各调节旋钮的交互强度 + 复合奖励                         |

## 9 愿景：研究路线图

Table 2 中的 WRP 矩阵暴露出两个结构上最稀疏的单元, 即 Router  $\times$  Pool 与 Pool  $\times$  Router; 同时, 在那些较为密集的单元里, 我们已有论文也已经识别出一些尚未被真正利用的交互。下面我们提出二十一个直接由这些空白推导出来的研究方向。每个方向都围绕这样一个问题展开: 语义路由器的信号架构, 以及 WRP 的跨维视角, 究竟独特地使什么事情成为可能。

Table 3 为每个机会标注了成熟度层级。在本节中, 凡是没有明确引用外部文献时, 所有预计收益都建立在我们既有测量结果之上。

### 9.1 Workload $\times$ Router

**开放机会 1** (面向最短会话长度路由的离线 RL). 在智能体工作负载中, 过长的会话往往是前几轮错误路由的后果。若路由器在第 2 轮选了较弱模型, 模型又在工具调用上失败, 智能体就会在第 3 轮带着更长提示去重试, 如此反复。一个若在起始阶段选对模型与工具即可在 3 轮内完成的会话, 最终可能拖到 8 轮, 而每多一轮都意味着更长提示和更多计算。

路由器会观察完整会话轨迹: 每轮用了哪个模型、调用了哪些工具、工具是否成功、整个会话经历了多少轮。这天然构成了离线 RL 数据集。状态可以表示为轮次编号、已尝试工具、累计上下

文长度与领域；动作可以表示为 (model, tool) 选择；奖励则可定义为负的会话长度，或质量加权后的任务完成度/会话长度。来自 [Section 4.4](#) 的工作负载身份信号还能零成本增强这一状态空间，例如 system prompt 指纹提供领域先验、工具定义暴露智能体运行模式、messages 数组深度直接给出会话成熟度。

---

**可行性与风险。** 路由器已经为 OATS 和 Feedback Detector 记录逐轮模型与工具结果，因此把这些记录组装成会话轨迹只是工程整理。真正的研究问题在于：如何做 credit assignment，如何进行 off-policy evaluation，以及它相对 OATS 的单步贪心策略究竟能把会话轮数与 token 数降到什么程度。

**开放机会 2** (HaluGate 驱动模型信誉路由)。Factcheck Classifier [9] 在请求阶段识别事实查询，HaluGate [8] 在响应阶段给出 token 级幻觉判断。今天，这些判决会被记录，但尚未反向作用于路由决策。

机会在于：路由器会长期积累 (model, domain, hallucinated) 三元组，因此可维护一个按模型、按领域组织的准确性估计。例如“模型 A 在医疗问题上 12% 会幻觉，在代码问题上只有 2%；模型 B 则相反”。这样，对每个新的事实查询，路由器就能以该领域上的历史幻觉率作为实时路由信号，在成本与延迟约束内优先选择更可靠模型。

---

**可行性与风险。** 数据收集没有新增开销，因为 HaluGate 本来就在运行。工程问题是样本量与时间漂移：每个 (model, domain) 单元需要多少判决才足够可靠，以及模型更新如何使历史信誉失效。贝叶斯估计和滑动窗口是自然方案。

**开放机会 3** (累积式多轮风险评分)。SafetyL1 [10] 和 SafetyL2 [11] 是按轮独立分类的，而多轮越狱攻击会把恶意意图拆散到多轮中，使得任何单轮都看起来不足以触发分类器。vLLM-SR [1] 通过对比嵌入对轮次序列进行判别，已经部分缓解了这一问题，但“检测到后如何做路由响应”尚未被严格形式化。

这里的机会有助于维护一个逐会话的累积风险分数，把每一轮的边缘性 SafetyL1/L2 信号累积起来。例如某一轮只有 0.4 的风险置信度，单独看不会被拦下，但连续三轮 0.4 就可能意味着真实升级。路由器可以对这些置信度做 EMA，当分数越过阈值时，逐步执行升级措施，例如切换到专门的安全模型、降低温度，或触发人工复核。

---

**可行性与风险。** SafetyL1 已经输出连续置信度，因此额外计算极小。难点在于阈值校准，尤其是如何避免对合法但触碰敏感话题的多轮对话产生误报。bearer token 还能把这种逐会话累积扩展到跨会话、跨智能体的逐调用者累积。

**开放机会 4** (基于路由器结果记忆的失败类别感知路由)。ErrorAtlas [72] 表明，每个 LLM 都有

独特的失败签名。当前生产路由器与 oracle 之间的差距，恰恰来自它们没有显式建模错误工具选择、错误参数、重试循环以及中途模型切换退化等失败模式。

vSR 已经通过 HaluGate、SafetyL1/L2、OATS 与 Feedback Detector 观察到每次请求的结果；再加上请求结构中透露出的工具集合和工具失败历史，它就可以构建一个按 *(model, domain, tool-set, failure class)* 组织的结果表。路由决策于是从“哪个模型最便宜/最准”变成“在这个领域与工具集合下，哪个模型最小化综合失败风险”。如果再加入 handoff penalty 和时间序列上的静默漂移检测，路由器就能避免高切换代价的模型组合，并对 provider 侧悄然发生的模型质量回退做出自动流量再平衡。

---

**可行性与风险。** 这些计数器与统计量大多可以零额外成本维护。真正开放的问题是：多维失败如何汇总成一个路由分数，表该细到什么粒度才不至于稀疏，以及漂移检测该多快触发而不误报。

**开放机会 5** (智能体会话的运行时令牌预算约束). 智能体工作流很容易发生 token 预算爆炸：单个用户请求会触发多次 LLM 调用，而且完整历史会在每轮反复重发。幻觉、错误工具选择和参数错误又会把无效内容注入上下文，使失败轮次在后续继续产生费用。更糟糕的是，定位故障步骤本身就不可靠，于是智能体常常只能退回到全上下文重摘要。

与智能体 SDK 内部的预算上限或事后结算层不同，路由器掌握逐轮真实 token 流、工具结果和历史会话成本分布，因此它才是最适合执行预算约束的地方。路由器可以维护每个活动会话的运行中 token 计数器，并借助 system prompt 指纹与消息深度来判断当前会话在所属部署中的“正常成本区间”。当某会话在当前轮次超过预期预算的 3 倍时，路由器可分级采取措施：先收缩工具目录，再压缩提示，再降级到更便宜模型，最后必要时直接终止。

---

**可行性与风险。** 计数器本身零开销，问题在于如何做领域特定预算模型的校准，并避免过早杀死那些本应很长但仍然合理的会话。逐调用者的跨会话预算也可由 bearer token 自然扩展。

**开放机会 6** (集群学习的联合工具-模型选择优化). 当前多数智能体框架在每次 LLM 调用时都发送完整工具目录，让同一个模型既负责选工具又负责消费所有工具结果。这在三个层面上都浪费：工具目录本身大量占用上下文；逐轮贪心工具选择并不关心完整序列的总成本与总收益；同一个模型并不适合所有工具调用。

这个机会要做的是，把工具选择与模型选择视为联合动作空间。路由器可以利用跨租户积累的 *(domain, turn-number, tool, model)* 结果表，在分发时同时做四件事：先依据权限缩减工具目录，再依据历史成功率暴露 top-k 工具，再为每个工具选择最合适的解析模型，最后结合序列级转移先验给出更符合当前阶段的偏好。



**可行性与风险。** 工具可学性与逐步模型可学性分别都已有大量工作验证。真正新的是：在路由层、用同一张集群级结果表，联合优化两者。主要风险是工具与模型交叉乘积带来的稀疏性，以及“集群先验”对新颖会话的适应性。

**开放机会 7** (网关协调的智能体循环：记忆层、工具与 talk-shaped 训练)。现有智能体 API 看似无状态，但实际上会在每轮重发庞大前缀、把工具输出持续塞入上下文，并在错误工具或错误模型选择后引发额外轮次。学术基准已经证明了这一问题。开放的 WRP 问题是：一个跨租户网关，是否能把动态工具表面、缓存/KV 策略、会话感知调度与拥塞控制统一起来，让那些根本没有重编译过的框架，也能获得更少 token、更少轮次和更高工具准确率。

这一机会的核心并不是发明单个新机制，而是把已经分别在 ITR、Continuum、AgServe、Helium、Sutradhara、CONCUR 等系统中验证过的子机制，统一到一个网关原生控制面中。路由器借助跨租户统计，可以不仅重写工具/指令表面，还能联合决策缓存块布局、KV TTL、接纳提示以及基于身份的记忆隔离与 delegation token。

---

**可行性与风险。** 这里最缺的不是机制，而是统一集成证据。需要在真实多租户流量上做 A/B，验证“组合这些机制”是否真的优于单独集成 ITR 或单独集成某种调度器。

## 9.2 Workload × Router：授权与安全

**开放机会 8** (面向智能体工具调用的请求级 RBAC 强制执行)。“Agents of Chaos” [68] 所展示的多种未授权工具执行攻击之所以成功，是因为模型既是决策者又是执行准入者，这违反了零信任中“每个资源请求都必须在执行点被独立认证与授权”的原则。路由器恰恰是那个同时知道 bearer token 身份、看得到模型 tool\_call 输出、又仍处于工具真正执行之前的唯一位置。

因此，路由器可以按 Kubernetes RBAC 的方式工作：先从 JWT 解出身份、角色和能力声明，再在治理 DSL 中定义按角色划分的工具权限矩阵，最后在模型做出工具调用时执行 block 或 rewrite。这样，未授权的调用者甚至在模型看到工具目录之前，就已经被收缩到安全范围之内。

---

**可行性与风险。** LiteLLM 的 Tool Permission Guardrail 与 AAP 已经分别提供了生产机制与 claim 结构。这里主要是工程集成工作，而不是基础原理问题。

**开放机会 9** (跨智能体威胁传播的全局调用者信誉)。在“Agents of Chaos”的实证场景里，同一个非 owner 会依次攻击多个智能体，而各智能体之间没有共享威胁记忆。路由器具备按 bearer token 聚合全局行为的能力，因此可以建立逐调用者的跨智能体行为画像：工具调用被拒、预算超限、安全升级、身份校验失败等事件都能被折算进一个带时间衰减的信誉分数中。

这样，当低信誉调用者接触一个新的智能体时，路由器就可以主动收缩工具目录、切换到更安全的模型、降低预算上限，并向 owner 发送侧信道通知。相反，安全交互则可随着时间逐步恢复信

任。

---

**可行性与风险。** 困难在于如何给异构安全事件统一定分，以及如何设置衰减与恢复速度，从而既对持久攻击者敏感，又不至于对短暂误判的正常用户过度惩罚。

**开放机会 10** (路由器强制执行的行为承诺)。在一些真实攻击案例中，模型文本上已经明确表示“我不应该继续回应”或“我无权这么做”，但用户继续施压后，它仍会重新参与。问题不在模型有没有表达出边界，而在系统没有把这段边界声明变成可持久执行的状态。

路由器可以把这种文本边界声明解析为一条结构化阻断规则，例如“bearer\_token\_X 对 agent\_Z 的某类操作在时长  $D$  内一律阻断”，从而使后续请求在进入模型之前就被拒绝。这样，模型的安全推理第一次真正拥有了“牙齿”。

---

**可行性与风险。** 最难的是区分真实边界承诺与普通措辞保留语，以及为阻断规则设置合适 TTL。它和调用者信誉体系之间也会相互作用。

### 9.3 Router $\times$ Pool

**开放机会 11** (Follow-the-sun 资源池再平衡)。FleetOpt [13] 在线下从静态 CDF 推导最优边界  $B^*$ ，但现实流量存在明显昼夜循环：白天偏聊天，夜间偏批量智能体。若系统能基于滑动窗口 CDF 周期性重算  $B^*$ ，并仅在软件层重新分配 vLLM 实例到短池或长池，就能让资源池在全天候流量迁移中始终靠近最优点。

---

**可行性与风险。** FleetOpt 的闭式求解已存在，真正要做的是把它包进一个滑动窗口控制器，并处理切换期间的暂时性 SLO 波动。

**开放机会 12** (从路由侧可观测量推断资源池状态)。当前 WRP 矩阵中的 Pool  $\times$  Router 单元仍是空白：现有工作几乎都没有把资源池状态反馈给路由决策。语义路由器今天主要只看请求本身的信号。

事实上，路由器在分发过程中已经拥有足够多的代理变量：每个实例的已分发请求数、已完成请求数、每个请求的 prompt length 以及最终的 TTFT。它可以据此估计队列深度、用 (prompt\_length, TTFT) 回归得到 prefill 速率，并据此在资源池拥堵时更激进压缩、换到更空闲的实例，或者避免再把长上下文请求发给状态差的实例。

**可行性与风险。** 队列估计近乎零成本；挑战在于冷启动、回归速度以及“线性回归 vs. 排队论估计”哪种更好。

**开放机会 13** (输出长度感知的资源池路由)。FleetOpt 只根据提示长度做资源池划分，但真正占用 KV-cache 的是提示与输出长度之和。比如代码智能体可能只有 3K 提示，却会生成 8K 输出，总计 11K token，更像长池工作负载。若只看提示长度，它就会被错误派往短池，引发溢出与 SLO 问题。

路由器可以利用领域先验和历史结果，对不同领域拟合输出长度分布或回归模型，从而在分发时用“prompt + predicted output”而不是只用 prompt 做资源池决策。

---

**可行性与风险。** 输出长度预测的波动性明显高于 TTFT，因此保守分位数策略很重要；过于激进会把太多请求送去长池，过于乐观则会压垮短池。

**开放机会 14** (资源池感知的级联)。传统级联只比较模型质量与成本，而不看强模型所在资源池是否已经拥塞。当强池队列过深时，把请求升级过去反而会造成 TTFT 违约。

借助 [机会 12](#) 的状态推断，路由器可以把级联决策变成“质量差距、强池队列、SLO 剩余空间”的联合函数。若强池已满，则可改为继续使用便宜模型、先压缩后重试，或改派中间层模型。

---

**可行性与风险。** 这里的核心研究问题是如何设置中止阈值与 grace period，从而在质量和延迟之间取得可接受的平衡。

**开放机会 15** (由路由器发出的 KV-cache 保留指令)。当智能体暂停去执行工具时，推理引擎必须暂存其 KV-cache。标准 LRU 并不知道“这个会话只是临时暂停，马上就会回来”，因此会把真正重要的缓存块驱逐掉，迫使系统重新计算 70K–200K token 的长上下文。

vLLM RFC 37003 [43] 提出了 RetentionDirective API，但没有解决“谁来决定优先级”这个策略问题。语义路由器正是天然的策略权威：会话链字段、system prompt 指纹、工具集合与 OATS 的工具时延分布，都能帮助它判断一个会话是否值得保留多久、以什么优先级保留。

---

**可行性与风险。** 这里主要是 API 打通与容量预算问题，即如何防止路由器把太多缓存都标成高优先级，最终反而让其他流量挨饿。

## 9.4 Workload × Pool

**开放机会 16** (由原型驱动的主动资源池扩缩容)。像 SageServe 与 WarmServe 这样的系统已经证明，工作负载预测可以让 GPU 扩缩容从被动变成主动。但它们大都预测聚合请求量，而不考

虑“流量语义组成”。

语义路由器掌握领域与原型分类，因此它能输出按 archetype 划分的时间序列。如果编码智能体流量正在上升而聊天流量正在下降，那么真正该提前扩容的是长池而不是短池。也就是说，工作负载的语义组成比总请求数更能精确地预测各资源池未来需求。

---

**可行性与风险。** 开放问题在于：这种 archetype-aware 预测，到底能否明显优于传统聚合计数预测。最自然的验证方式是把它放到 FleetSim 的昼夜模拟流量中做对比实验。

## 9.5 三维：Workload $\times$ Router $\times$ Pool

**开放机会 17** (压缩率与资源池边界的在线联合自适应). **机会 11** 会随着工作负载 CDF 变化而调整  $B^*$ ，但默认把压缩率  $\gamma$  固定不变。FleetOpt 已经证明两者本来就是耦合的：最佳压缩率依赖于资源池边界，最佳资源池边界也依赖于压缩强度。

因此，一个更完整的控制器应当每隔固定时间，在滑动窗口 CDF 上同时搜索  $(\gamma, B^*)$  的最优组合，并只在收益足够明显时通过迟滞机制更新配置。对于聊天主导与智能体主导的不同原型，最佳压缩策略本就不同。

---

**可行性与风险。** 网格搜索开销很低，真正问题是如何避免频繁振荡，以及联合优化相对“只调边界”到底有多大额外收益。

**开放机会 18** (混合原型的集群配置). FleetOpt 与 FleetSim 通常面向单一 CDF 原型做优化，而生产集群往往同时服务客服聊天、内部 RAG 与编码智能体等多种原型。语义路由器能实时给出各原型的占比，因此它正好提供了扩展 FleetOpt 到“加权混合 CDF”所缺失的输入。

一个 60% Archetype 1 与 40% Archetype 3 混合的集群，很可能比“分别为最坏单原型配置”更节省，因为短池与长池的压力在时间与资源维度上互补。

---

**可行性与风险。** 关键研究问题是：这种互补性到底强不强，是否足以抵消引入混合配置的复杂度；另一个风险则是原型分类本身若不准，会让有效 CDF 偏移。

**开放机会 19** (把能耗作为可组合路由目标). 1/W 定律 [15] 已经给出了与上下文长度、资源池配置和 GPU 类型有关的闭式 tok/W 模型，但它目前主要还是离线分析工具，尚未成为请求时路由信号。

这一机会就是把 1/W 模型真正操作化为 vSR 里的一个新信号类型，使路由器在比较候选资源池时，不只看成本、延迟和质量，也看能耗。这样，运营者就可以定义诸如“整个集群 tok/W 不

低于某阈值”的能效 SLO。

---

**可行性与风险。** 模型本身很快，也已有研究开始探索能耗/碳感知的 LLM 路由。最大的开放问题是，多目标之间应如何权衡，尤其在成本与能耗并不完全一致时。

**开放机会 20** (面向 WRP 架构的治理即代码). vLLM-SR DSL [1] 与 SIRP [20] 已经提供了 policy-as-code 的雏形。若进一步把 DSL 扩展到能表达 WRP 级约束，例如“PII 查询必须留在本地机房”“智能体工作负载必须走高准确率资源池”“每请求成本不得超过某个上限”，那么整个推理架构的治理就可以编译化、静态化。

bearer token 进一步把这套 DSL 从“资源池、区域、成本”等基础设施属性，扩展到“角色、scope、授权声明”等调用者属性，从而让 RBAC 谓词成为治理语言的一部分，并与 机会 8 的运行时执行形成前后配合。

---

**可行性与风险。** 这项工作的核心挑战在于语言设计、编译器诊断以及“策略可满足性”验证，尤其是随着集群不断演化，已有策略如何持续保持可满足。

## 9.6 顶层：闭环自适应

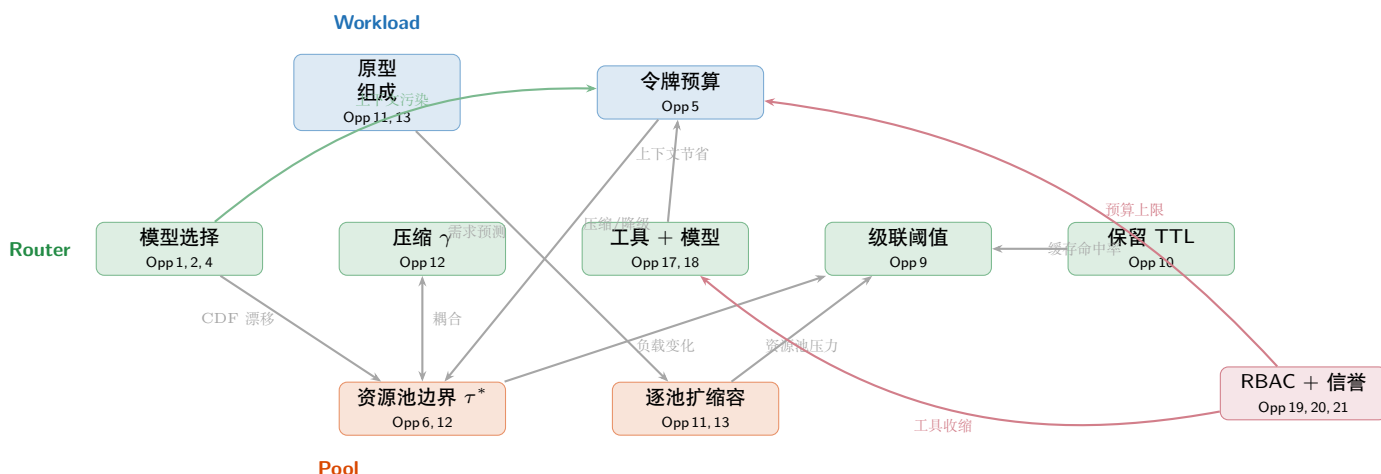
**开放机会 21** (跨 WRP 调节旋钮的闭环自适应). 前面的二十个机会各自围绕某个或某几个调节旋钮展开，例如模型选择、压缩率  $\gamma$ 、资源池边界  $B^*$ 、保留 TTL、级联阈值、令牌预算、联合工具-模型选择、安全升级阈值、RBAC 规则、调用者信誉与行为承诺阻断。单独看时，每个回路都说得通；但一旦放到一起，它们就会互相影响。改变模型选择会改变会话长度分布，进而改变工作负载 CDF，进而影响最佳资源池边界，再进一步影响级联是否值得发生。

因此，最终机会是构建一个元控制器，以 MAPE-K 的方式统一编排所有自适应回路：监控统一的逐会话结果记录，分析失败类别及其对应的责任旋钮，规划满足交互约束的联合调整，再以离线估计 + 小流量 canary 的方式逐步执行。与单独微调每个旋钮相比，这种闭环系统才真正符合 WRP 的耦合本质。

---

**可行性与风险。** MAPE-K 本身很成熟，数据库调优和云系统已经证明了多旋钮安全在线调节的可行性。这里真正开放的问题是交互强度是否足够大，值得引入一个统一元控制器；以及如何用离线日志先评估候选策略，避免把每次尝试都直接暴露给线上流量。





**Figure 2** WRP 各机会中的调节旋钮依赖关系。调整某个旋钮，例如模型选择，会沿着工作负载 CDF 一路传导到资源池边界与级联阈值，因此需要协调式自适应。

## 10 更广泛的背景：相关工作

综述。Moslem 和 Kelleher [102] 综述了动态模型路由与级联，覆盖查询难度、人类偏好、聚类、不确定性、RL 与多模态路由；但他们关于何时/什么/如何的分类，并未涉及资源池架构或工作负载交互。Pan 与 Li [103] 覆盖了从算子优化到无服务器部署的完整推理栈，但把路由视为集群级负载均衡。Xia *et al.* [104] 则从实例级和集群级两个层面综述高效 LLM 推理服务。这些综述都没有形式化 WRP 框架所关注的跨维交互。

路由系统（不含我们的项目）。RouteLLM [50] 与 Hybrid LLM [105] 使用偏好数据在强/弱模型之间做路由。FrugalGPT [88] 与 AutoMix [89] 使用级联。Router-R1 [53] 与 R2-Router [54] 使用基于 RL 的选择。MixLLM [51] 在 NAACL 2025 上使用上下文 bandit。xRouter [62] 则在 20+ LLM 工具上训练成本感知 RL 路由器，并显式使用成本-性能奖励。

智能体服务与失败分析。AGSERVE [40] 为智能体工作负载提供会话感知的 KV-cache 管理与模型级联（NeurIPS 2025）。Helium [56] 把智能体工作流当作查询计划来处理，并引入主动缓存和缓存感知调度。SUTRADHARA [57] 围绕工具型智能体，对编排器与服务引擎做联合设计。Continuum [41] 在 SWE-Bench 和 BFCL 上为多轮智能体调度引入 KV-cache TTL pinning。Concur [58] 通过基于拥塞的并发调节，把智能体级接纳控制引入系统（ICML 2026）。与此同时，越来越多的经验研究开始分类智能体故障模式 [74]（从 13,602 个问题中归纳出 37 种故障类型）、模型特定失败特征 [72]（83 个模型、35 个数据集），以及多轮模型切换退化 [76]（一次切换即可带来  $\pm 4$ –13 个百分点波动）；但这些工作都没有把失败信号反馈回路由决策之中。

集群配置（不含我们的项目）。Mélange [63] 优化异构 GPU 分配。SageServe [52] 则在微软规模上把流量预测整合进来。

解耦推理。Splitwise [23]、DistServe [44] 与 Sarathi-Serve [106] 把 prefill 与 decode 解耦。Mooncake [64] 提供以 KV-cache 为中心的解耦。NVIDIA Dynamo [65] 实现了动态多节点调度。

能效（不含我们的项目）。SweetSpot [107] 提供分析式能耗预测。TokenPowerBench [108] 与 GreenServ [94] 则对推理能耗进行基准测试并提出优化方法。

## 11 结论

我们论证了，LLM 推理优化这一版图最适合围绕三个耦合维度来组织，即 Workload、Router 与 Pool，而真正支配现实性能的正是这些维度之间的相互作用。这一论证建立在 vLLM Semantic Router 项目不断扩展的论文体系之上，涵盖了信号组合、冲突检测、资源池路由、集群配置、能耗建模、多模态智能体路由、工具选择、安全、语义缓存、反馈驱动的自适应、幻觉检测、低延迟嵌入模型，以及用于隐私保护和越狱防护的分层内容安全分类。我们反复看到，只要解决其中一个维度里的问题，就会暴露出它对另一个维度的依赖。

WRP 矩阵表明，最大的开放问题位于各个交叉处，尤其是稀疏的 Router  $\times$  Pool 与 Pool  $\times$  Router 单元。我们提出的二十一项机会（Table 3）正是为了填补这些空缺，并且都立足于语义路由器这一独特视角：信号-决策架构已经能够按领域、模态和复杂度对每个请求进行分类，通过 HaluGate 验证响应，按实例跟踪分发与完成事件，并生成逐轮安全置信度分数；这些都是尚未被充分利用的输入，可用于模型选择、集群配置、级联策略与能耗优化。

其中有六项属于工程层项目，可以直接由现有组件构建出来：基于 HaluGate 的模型信誉路由（机会 2）、运行时令牌预算约束（机会 5）、follow-the-sun 式资源池再平衡（机会 11）、由路由器发出的 KV-cache 保留指令（机会 15）、治理即代码（机会 20）以及请求级 RBAC 强制执行（机会 8）。其余十五项属于研究层：面向会话长度的离线 RL 路由（机会 1）、累积式多轮风险评分（机会 3）、基于路由器结果记忆的失败类别感知路由（机会 4）、集群级学习的联合工具-模型优化（机会 6）、由网关协调的智能体循环、记忆层与工具表面（机会 7）、跨智能体的全局调用者信誉（机会 9）、由路由器执行的行为承诺（机会 10）、资源池状态推断（机会 12）、输出长度感知的资源池路由（机会 13）、资源池感知级联（机会 14）、在线  $(\gamma, \mathcal{B}^*)$  联合自适应（机会 17）、混合原型集群配置（机会 18）、由原型驱动的主动资源池扩缩容（机会 16）、把能耗作为路由目标（机会 19），以及跨 WRP 参数的闭环自适应（机会 21）。

随着生产工作负载从聊天逐步转向智能体与多模态应用，本文识别出的这些跨维交互只会变得更加重要。我们希望 WRP 框架，以及它对“哪些东西已经可以做、哪些问题仍然开放”的判断，能够对从事这一方向的人有所帮助。

## References

- [1] vLLM Semantic Router Team. vLLM semantic router: Signal driven decision routing for mixture-of-modality models. *arXiv preprint arXiv:2603.04444*, 2026.

- [2] Xunzhuo Liu, Hao Wu, Huamin Chen, Bowei He, and Xue Liu. Conflict-free policy languages for probabilistic ML predicates: A framework and case study with the semantic router DSL. *arXiv preprint arXiv:2603.18174*, 2026.
- [3] Xunzhuo Liu, Bowei He, Xue Liu, Andy Luo, Haichen Zhang, and Huamin Chen. 98× faster LLM routing without a dedicated GPU: Flash attention, prompt compression, and near-streaming for the vLLM semantic router. *arXiv preprint arXiv:2603.12646*, 2026.
- [4] vLLM Semantic Router Team. mmBERT-embed-32k-2d-matryoshka: Multilingual embedding model with 2d matryoshka training. Hugging Face model: `llm-semantic-router/mmbert-embed-32k-2d-matryoshka`, 2025.
- [5] Chen Wang, Xunzhuo Liu, Yuhan Liu, Yue Zhu, Xiangxi Mo, Junchen Jiang, and Huamin Chen. When to reason: Semantic router for vLLM. In *NeurIPS Workshop on ML for Systems (MLForSys)*, 2025.
- [6] Chen Wang, Xunzhuo Liu, Yue Zhu, Alaa Youssef, Priya Nagpurkar, and Huamin Chen. Category-aware semantic caching for heterogeneous LLM workloads. *arXiv preprint arXiv:2510.26835*, 2025.
- [7] vLLM Semantic Router Team. mmBERT-32k feedback detector: User satisfaction classification for online routing adaptation. Hugging Face model: `llm-semantic-router/mmbert32k-feedback-detector-lora`, 2026.
- [8] vLLM Semantic Router Team. Token-level truth: Real-time hallucination detection for production LLMs. vLLM Blog, 2025. <https://blog.vllm.ai/2025/12/14/halugate.html>.
- [9] vLLM Semantic Router Team. mmBERT-32k factcheck classifier: Binary prompt classification for conditional hallucination detection. Hugging Face model: `llm-semantic-router/mmbert32k-factcheck-classifier-merged`, 2026.
- [10] vLLM Semantic Router Team. MLCommons AI safety classifier – level 1 (binary): Safe vs. unsafe content classification. Hugging Face model: `llm-semantic-router/mlcommons-safety-classifier-level1-binary`, 2026.
- [11] vLLM Semantic Router Team. MLCommons AI safety classifier – level 2 (9-class hazard): Hierarchical content safety classification. Hugging Face model: `llm-semantic-router/mlcommons-safety-classifier-level2-hazard`, 2026.
- [12] Huamin Chen, Xunzhuo Liu, Yuhan Liu, Junchen Jiang, Bowei He, and Xue Liu. Token-budget-aware pool routing for cost-efficient LLM inference. *arXiv preprint*, 2026.
- [13] Huamin Chen, Xunzhuo Liu, Yuhan Liu, Junchen Jiang, Bowei He, and Xue Liu. FleetOpt: Analytical fleet provisioning for LLM inference with compress-and-route as implementation mechanism. *arXiv preprint arXiv:2603.16514*, 2026.
- [14] Huamin Chen, Xunzhuo Liu, Yuhan Liu, Junchen Jiang, Bowei He, and Xue Liu. inference-fleet-sim: A queueing-theory-grounded fleet capacity planner for LLM inference. *arXiv preprint arXiv:2603.16054*, 2026.
- [15] Huamin Chen, Xunzhuo Liu, Yuhan Liu, Junchen Jiang, Bowei He, and Xue Liu. The 1/W law: An analytical study of context-length routing topology and GPU generation gains for LLM inference energy efficiency. *arXiv preprint arXiv:2603.17280*, 2026.
- [16] Xunzhuo Liu, Bowei He, Xue Liu, Andy Luo, Haichen Zhang, and Huamin Chen. Adaptive vision-language model routing for computer use agents. *arXiv preprint arXiv:2603.12823*, 2026.

- [17] Huamin Chen, Xunzhuo Liu, Junchen Jiang, Bowei He, and Xue Liu. Outcome-aware tool selection for semantic routers: Latency-constrained learning without LLM inference. *arXiv preprint arXiv:2603.13426*, 2026.
- [18] Xunzhuo Liu, Bowei He, Xue Liu, Andy Luo, Haichen Zhang, and Huamin Chen. Visual confused deputy: Exploiting and defending perception failures in computer-using agents. *arXiv preprint arXiv:2603.14707*, 2026.
- [19] OpenClaw contributors. OpenClaw: Personal AI assistant with a local-first gateway. Open-source software (MIT License), 2026. Repository: <https://github.com/openclaw/openclaw>. Gateway Web-Socket control plane for multi-channel agent sessions, tools, and model routing; documentation at <https://docs.openclaw.ai>.
- [20] Huamin Chen and Luay Jalil. Semantic inference routing protocol (SIRP). Internet Engineering Task Force (IETF), 2025.
- [21] Huamin Chen, Luay Jalil, and N. Cocker. Multi-provider extensions for agentic AI inference APIs. Internet Engineering Task Force (IETF), Network Management Research Group, 2025.
- [22] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, et al. LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. In *Proceedings of ICLR*, 2024.
- [23] Pratyush Patel, Esha Choukse, Chaojie Zhang, et al. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of ISCA*, 2024.
- [24] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of ICLR*, 2024.
- [25] Shishir G. Patil, Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of ICML*, 2025.
- [26] Yuxing Xiang, Xue Li, Kun Qian, Wenyuan Yu, Ennan Zhai, and Xin Jin. ServeGen: Workload characterization and generation of large language model serving in production. *arXiv preprint arXiv:2505.09999*, 2025.
- [27] Han Bao, Zheyuan Zhang, Pengcheng Jing, et al. Drift-bench: Diagnosing cooperative breakdowns in LLM agents under input faults via multi-turn interaction. *arXiv preprint arXiv:2602.02455*, 2026.
- [28] Xuannan Liu, Xiao Yang, Zekun Li, et al. AgentHallu: Benchmarking automated hallucination attribution of LLM-based agents. *arXiv preprint arXiv:2601.06818*, 2026.
- [29] Pengfei Du. Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*, 2026.
- [30] Natchanon Pollertlam and Witchayut Kornsuwanawit. Beyond the context window: A cost-performance analysis of fact-based memory vs. long-context LLMs for persistent agents. *arXiv preprint arXiv:2603.04814*, 2026.
- [31] Minki Kang, Wei-Ning Chen, Dongge Han, et al. ACON: Optimizing context compression for long-horizon LLM agents. *arXiv preprint arXiv:2510.00615*, 2025.
- [32] Nikhil Verma. Active context compression: Autonomous memory management in LLM agents. *arXiv preprint arXiv:2601.07190*, 2026.

- [33] Yubin Ge, Salvatore Romeo, Jason Cai, et al. SAMULE: Self-learning agents enhanced by multi-level reflection. In *Proceedings of EMNLP*, 2025. arXiv:2509.20562.
- [34] Yifan Yu, Moyan Li, Shaoyuan Xu, et al. CORRECT: COndensed eRror RECOgnition via knowledge transfer in multi-agent systems. *arXiv preprint arXiv:2509.24088*, 2025.
- [35] Xuanbo Su, Yingfang Zhang, Hao Luo, et al. Mistake notebook learning: Batch-clustered failures for training-free agent adaptation. *arXiv preprint arXiv:2512.11485*, 2025.
- [36] Uria Franko. Dynamic system instructions and tool exposure for efficient agentic LLMs. *arXiv preprint arXiv:2602.17046*, 2026.
- [37] Marianne Menglin Liu, Daniel Garcia, Fjona Parllaku, Vikas Upadhyay, Syed Fahad Allam Shah, and Dan Roth. ToolScope: Enhancing LLM agent tool use through tool merging and context-aware filtering. *arXiv preprint arXiv:2510.20036*, 2025.
- [38] Cheng Qian, Emre Can Acikgoz, Hongru Wang, et al. SMART: Self-aware agent for tool overuse mitigation. *arXiv preprint arXiv:2502.11435*, 2025.
- [39] Tengxiao Liu, Zifeng Wang, Jin Miao, et al. Budget-aware tool-use enables effective agent scaling. *arXiv preprint arXiv:2511.17006*, 2025.
- [40] Yanyu Ren, Li Chen, Dan Li, et al. Transcending cost-quality tradeoff in agent serving via session-awareness. In *NeurIPS*, 2025.
- [41] Hanchen Li et al. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. *arXiv preprint arXiv:2511.02230*, 2025.
- [42] llm-d Team. KV-Cache wins you can see: From prefix caching in vLLM to distributed scheduling with llm-d. <https://llm-d.ai/blog/kvcache-wins-you-can-see>, 2026.
- [43] vLLM Community. RFC: Context-aware KV-cache retention API (prioritized evictions). <https://github.com/vllm-project/vllm/issues/37003>, 2026.
- [44] Yinmin Zhong, Shengyu Liu, Junda Chen, et al. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of OSDI*, 2024.
- [45] BerriAI. LiteLLM tool permission guardrail. [https://docs.litellm.ai/docs/proxy/guardrails/tool\\_permission](https://docs.litellm.ai/docs/proxy/guardrails/tool_permission), 2026.
- [46] AAP Protocol Working Group. Agent authorization profile (AAP): OAuth 2.0 extension for agent authorization. <https://www.aap-protocol.org/>, 2026.
- [47] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. Zero trust architecture. Technical Report SP 800-207, National Institute of Standards and Technology, 2020.
- [48] Ryan Marinelli, Josef Pichlmeier, and Tamas Bisztray. Harnessing chain-of-thought metadata for task routing and adversarial prompt detection. *arXiv preprint arXiv:2503.21464*, 2026.
- [49] Yinwei Dai, Zhuofu Chen, Anand Iyer, et al. Aragog: Just-in-time model routing for scalable serving of agentic workflows. *arXiv preprint arXiv:2511.20975*, 2025.
- [50] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, et al. RouteLLM: Learning to route LLMs with preference data. In *Proceedings of ICLR*, 2025.
- [51] Renxi Lu et al. MixLLM: Dynamic contextual-bandit-based routing for multi-LLM systems. In *Proceedings of NAACL*, 2025.



- [52] Shashwat Jaiswal, Kunal Jain, Yogesh Simmhan, et al. SageServe: Optimizing LLM serving on cloud data centers with forecast aware auto-scaling. *arXiv preprint arXiv:2502.14617*, 2025.
- [53] Haozhen Zhang, Tao Feng, and Jiaxuan You. Router-R1: Teaching LLMs multi-round routing and aggregation via reinforcement learning. In *Proceedings of NeurIPS*, 2025. arXiv:2506.09033.
- [54] Jiaqi Xue, Qian Lou, Jiarong Xing, and Heng Huang. R2-Router: A new paradigm for LLM routing with reasoning. *arXiv preprint arXiv:2602.02823*, 2026.
- [55] Xianzhi Zhang, Yue Xu, Yinlin Zhu, Di Wu, Yipeng Zhou, Miao Hu, and Guocong Quan. Adapter-augmented bandits for online multi-constrained multi-modal inference scheduling. *arXiv preprint arXiv:2603.06403*, 2026.
- [56] Noppanat Wadlom, Junyi Shen, and Yao Lu. Efficient LLM serving for agentic workflows: A data systems perspective. *arXiv preprint arXiv:2603.16104*, 2026.
- [57] Anish Biswas, Kanishk Goel, Jayashree Mohan, et al. Sutradhara: An intelligent orchestrator-engine co-design for tool-based agentic inference. *arXiv preprint arXiv:2601.12967*, 2026.
- [58] Qiaoling Chen, Zhisheng Ye, Tian Tang, et al. CONCUR: High-throughput agentic batch inference of LLM via congestion-based concurrency control. *arXiv preprint arXiv:2601.22705*, 2026. ICML 2026.
- [59] Guibin Zhang, Haiyang Yu, Kaiming Yang, et al. EvoRoute: Experience-driven self-routing LLM agent systems. *arXiv preprint arXiv:2601.02695*, 2026.
- [60] Caiqi Zhang, Menglin Xia, Xuchao Zhang, Daniel Madrigal, Ankur Mallick, Samuel Kessler, Victor Ruehle, and Saravan Rajmohan. Budget-aware agentic routing via boundary-guided training. *arXiv preprint arXiv:2602.21227*, 2026.
- [61] Elias Lumer, Faheem Nizar, Akshaya Jangiti, et al. Don’t Break the Cache: An Evaluation of Prompt Caching for Long-Horizon Agentic Tasks. *arXiv preprint arXiv:2601.06007*, 2026.
- [62] Cheng Qian, Zuxin Liu, Shirley Kokane, et al. xRouter: Training cost-aware LLMs orchestration system via reinforcement learning. *arXiv preprint arXiv:2510.08439*, 2025.
- [63] Tyler Griggs, Xiaoxuan Liu, Jiaxiang Yu, Doyoung Kim, and Enze Zhong. Mélange: Cost efficient large language model serving by exploiting GPU heterogeneity. *arXiv preprint arXiv:2404.14527*, 2024.
- [64] Ruoyu Qin et al. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *arXiv preprint arXiv:2407.00079*, 2025.
- [65] NVIDIA. NVIDIA dynamo: Smart multi-node scheduling for LLM inference. <https://developer.nvidia.com/blog/smart-multi-node-scheduling-for-fast-and-efficient-llm-inference-with-nvidia-runai-and-nv>, 2026.
- [66] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of SOSP*, 2023.
- [67] Jesse van Remmerden, Zaharah Bukhsh, and Yingqian Zhang. Generalizing beyond suboptimality: Offline reinforcement learning learns effective scheduling through random data. *arXiv preprint arXiv:2509.10303*, 2025.
- [68] Erica Barua, Pradeep Dasigi, and Muru Zhang. Agents of chaos: Evaluating LLM agent vulnerabilities through real-world interactions. *arXiv preprint arXiv:2602.20021*, 2025.
- [69] Mark Russinovich, Ahmed Salem, and Ronen Eldan. Crescendo: Escalating multi-turn LLM jailbreak attacks. *arXiv preprint arXiv:2404.01833*, 2024. USENIX 2025.

- [70] J. Alex Corll. Peak + accumulation: A proxy-level scoring formula for multi-turn LLM attack detection. *arXiv preprint arXiv:2602.11247*, 2026.
- [71] Justin Albrethsen, Yash Datta, Kunal Kumar, et al. DeepContext: Stateful multi-turn jailbreak detection via temporal intent drift. *arXiv preprint arXiv:2602.16935*, 2026.
- [72] Shir Ashury-Tahan, Yifan Mai, Elron Bandel, et al. ErrorMap and ErrorAtlas: Charting the failure landscape of large language models. *arXiv preprint arXiv:2601.15812*, 2026.
- [73] Hao Li, Yiqun Zhang, Zhaoyan Guo, et al. LLMRouterBench: A massive benchmark and unified framework for LLM routing. *arXiv preprint arXiv:2601.07206*, 2026.
- [74] Mehil B. Shah, Mohammad Mehdi Morovati, Mohammad Masudur Rahman, et al. Characterizing faults in agentic AI: A taxonomy of types, symptoms, and root causes. *arXiv preprint arXiv:2603.06847*, 2026.
- [75] Kunlun Zhu, Zijia Liu, Bingxuan Li, et al. PALADIN: Training LLM agents to recover from tool-call errors via systematic failure injection. *arXiv preprint arXiv:2509.25370*, 2025.
- [76] Tianjun Gao et al. Measuring multi-turn model-switch performance drift in LLM serving. *ICLR 2026 CAO Workshop*, 2026.
- [77] Philipp Steiner, Ralph Peeters, and Christian Bizer. MCP vs RAG vs NLWeb vs HTML: A comparison of the effectiveness and efficiency of different agent interfaces to the web. In *Proceedings of The Web Conference*, 2026. *arXiv:2511.23281*.
- [78] Jingyi Jia and Qinbin Li. AutoTool: Efficient tool selection for large language model agents. *arXiv preprint arXiv:2511.14650*, 2025.
- [79] Sami Abuzakuk, Anne-Marie Kermarrec, Rishi Sharma, et al. Optimizing agentic workflows using meta-tools. *arXiv preprint arXiv:2601.22037*, 2026.
- [80] Pengbo Liu. ToolRLA: Multiplicative reward decomposition for tool-integrated agents. *arXiv preprint arXiv:2603.01620*, 2026.
- [81] Nikunj Gupta et al. HierRouter: Coordinated routing of specialized large language models via reinforcement learning. *arXiv preprint arXiv:2511.09873*, 2025.
- [82] Cloud Native Computing Foundation. Open policy agent. <https://openpolicyagent.org>, 2024. CNCF Graduated Project.
- [83] Krzysztof Rządca et al. Autopilot: Workload autoscaling at Google scale. In *Proceedings of EuroSys*, 2020.
- [84] Inho Cho et al. Overload control for  $\mu$ s-scale RPCs with Breakwater. In *Proceedings of OSDI*, 2020.
- [85] Minghao Yang et al. TopFull: An adaptive top-down overload control for SLO-oriented microservices. In *Proceedings of ASPLOS*, 2024.
- [86] Shashank Swamy et al. ALPS: Activation-based length prediction for intelligent LLM inference scheduling. *Google Research*, 2025.
- [87] Huanyi Xie, Yubin Chen, Liangyu Wang, et al. Predicting LLM output length via entropy-guided representations. *arXiv preprint arXiv:2602.11812*, 2026.
- [88] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- [89] Pranjal Aggarwal, Aman Madaan, Ankit Anand, et al. AutoMix: Automatically mixing language models. *arXiv preprint arXiv:2310.12963*, 2024.

- [90] Chiheng Lou, Sheng Qi, Rui Kang, et al. WarmServe: Enabling one-for-many GPU prewarming for multi-LLM serving. *arXiv preprint arXiv:2512.09472*, 2025.
- [91] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of EuroSys*, 2015.
- [92] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. Multi-resource packing for cluster schedulers. In *Proceedings of ACM SIGCOMM*, 2014.
- [93] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *Proceedings of NSDI*, 2011.
- [94] Thomas Ziller, Shashikant Ilager, Alessandro Tundo, Ezio Bartocci, Leonardo Mariani, and Ivona Brandic. GreenServ: Energy-efficient context-aware dynamic routing for multi-model LLM inference. *arXiv preprint arXiv:2601.17551*, 2026.
- [95] Seongjin Park et al. GAR: Carbon-aware routing for LLM inference via constrained optimization. *ICLR 2026 Workshop*, 2026.
- [96] Walid A. Hanafy, Li Wu, David Irwin, et al. CarbonFlex: Enabling carbon-aware provisioning and scheduling for cloud clusters. *arXiv preprint arXiv:2505.18357*, 2025.
- [97] Xinyi Zhang et al. Towards dynamic and safe configuration tuning for cloud databases. *arXiv preprint arXiv:2203.14473*, 2022. SIGMOD 2024.
- [98] Jeffrey O. Kephart and David M. Chess. The vision of autonomic computing. *Computer*, 36(1):41–50, 2003.
- [99] Olivier Jeunen and Aleksei Ustimenko.  $\delta$ -OPE: Off-policy estimation with pairs of policies. *arXiv preprint arXiv:2405.10024*, 2024.
- [100] Amit Singh Bhatti, Vishal Vaddina, and Dagnachew Birru. PROTEUS: SLA-aware routing via Lagrangian RL for multi-LLM serving systems. *arXiv preprint arXiv:2601.19402*, 2026.
- [101] Wang Wei, Tiankai Yang, Hongjie Chen, et al. BaRP: Bandit-feedback routing with preferences for multi-LLM inference. *arXiv preprint arXiv:2510.07429*, 2025.
- [102] Yasmin Moslem and John D. Kelleher. Dynamic model routing and cascading for efficient LLM inference: A survey. *arXiv preprint arXiv:2603.04445*, 2026.
- [103] James Pan and Guoliang Li. A survey of LLM inference systems. *arXiv preprint arXiv:2506.21901*, 2025.
- [104] Chunyu Xia et al. Taming the titans: A survey of efficient LLM inference serving. In *Proceedings of INLG*, 2025.
- [105] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, et al. Hybrid LLM: Cost-efficient and quality-aware query routing. *arXiv preprint arXiv:2404.14618*, 2024.
- [106] Amey Agrawal, Nitin Kedia, Ashish Panwar, et al. Sarathi-Serve: Efficient LLM inference by piggybacking decodes with chunked prefills. In *Proceedings of OSDI*, 2024.
- [107] Hiari Pizzini Cavagna, Andrea Proia, Giacomo Madella, et al. SweetSpot: An analytical model for predicting energy efficiency of LLM inference. *arXiv preprint arXiv:2602.05695*, 2026.
- [108] Chenxu Niu, Wei Zhang, Jie Li, Yongjian Zhao, Tongyang Wang, Xi Wang, and Yong Chen. Token-PowerBench: Benchmarking the power consumption of LLM inference. *arXiv preprint arXiv:2512.03024*, 2025.